# Tablas de direccionamiento directo y tablas hash

## Buscar sin recorrer, con pocas o con muchas claves posibles

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

# Objetivos de aprendizaje I

## Al finalizar esta sesión, el estudiante podrá

- Definir el TAD Diccionario e implementarlo con una tabla de direccionamiento directo, con el costo en tiempo y en espacio de cada operación (OA1, OA2, OA3).
- Reconocer cuándo el universo de claves hace inviable la tabla directa (OA1).
- Calcular la casilla de una clave entera o de una cadena con el método de la división y reconocer cuándo dos claves chocan (OA2).
- Insertar, buscar y eliminar a mano y en C++ en una tabla hash con encadenamiento (OA2, OA3).
- Dar el costo de cada operación en función del factor de carga $\alpha = n/m$ y escoger el número de casillas $m$ (OA1, OA5).

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# ¿Quién entró al laboratorio? I

**Problema**

En la puerta de un laboratorio se registra a cada estudiante que entra, con un código entre 0 y 999 y su nombre. Durante el día llegan miles de preguntas: ¿entró el 412? Y a veces se retira un registro. ¿Cómo se responde cada pregunta sin revisar todos los registros?

# ¿Entendimos el problema? I

## Entradas

Tres clases de orden: registrar un código con un nombre, preguntar por un código y retirar un código. Un código no se registra dos veces; si llega otra vez, cambia el nombre.

## Salida

Por cada pregunta, el nombre registrado con ese código o `no esta`.

## Con códigos de 0 a 9

Se registran 3 Ana, 7 Luis y 0 Eva. Se pregunta por el 7 y por el 5, se retira el 3 y se pregunta por el 3.

## Pregunta

¿Qué se imprime en cada una de las tres preguntas?

# ¿Entendimos el problema? II

| Orden | Registrados | Imprime |
|---|---|---|
| registrar 3 Ana | 3 Ana | |
| registrar 7 Luis | 3 Ana, 7 Luis | |
| registrar 0 Eva | 3 Ana, 7 Luis, 0 Eva | |
| preguntar 7 | | `buscar(7): Luis` |
| preguntar 5 | | `buscar(5): no esta` |
| retirar 3 | 7 Luis, 0 Eva | |
| preguntar 3 | | `buscar(3): no esta` |

El 3 estuvo registrado, pero se retiró antes de la pregunta.

# Primera idea: una lista I

Los registros en una `list`, en orden de llegada. Preguntar camina desde el comienzo hasta encontrar el código.

```
// Theta(n) en el peor caso: camina hasta encontrar la clave o hasta
↪  el final
list<Registro>::iterator buscar(list<Registro> &l, int clave) {
  list<Registro>::iterator it = l.begin();
  while (it != l.end() && it->clave != clave) {
    ++it;
  }
  return it;
}
```




`buscar(5)`: tres pasos y no está

# Primera idea: una lista II

| Operación | Tiempo | Por qué |
|---|---|---|
| registrar | $\Theta(1)$ | `push_back`, si no se revisa que el código ya esté |
| preguntar | $\Theta(n)$ en el peor caso | camina nodo a nodo; un código que no está recorre los $n$ |
| retirar | $\Theta(n)$ en el peor caso | buscar el nodo; `erase` ya es $\Theta(1)$ |

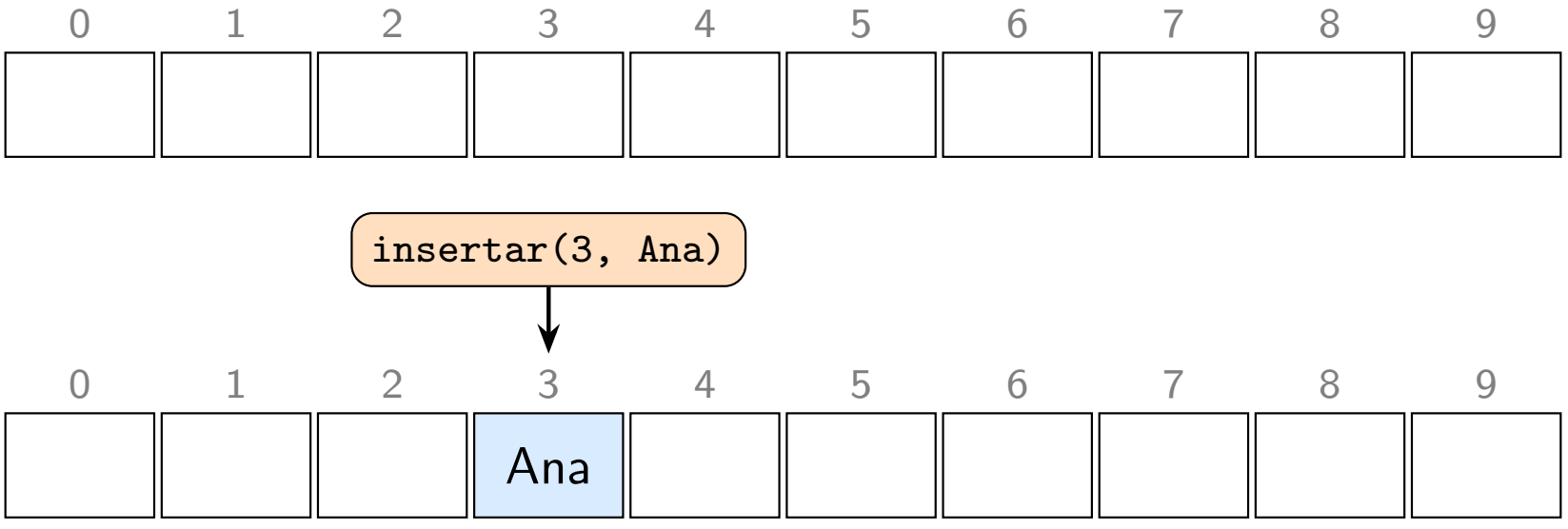
Con 800 registrados y 10 000 preguntas por códigos que no están, se recorren 8 000 000 nodos. El espacio es $\Theta(n)$.

**Pregunta**

El código ya es un número entre 0 y 999. ¿Hace falta buscarlo, o puede decir él mismo dónde está guardado?

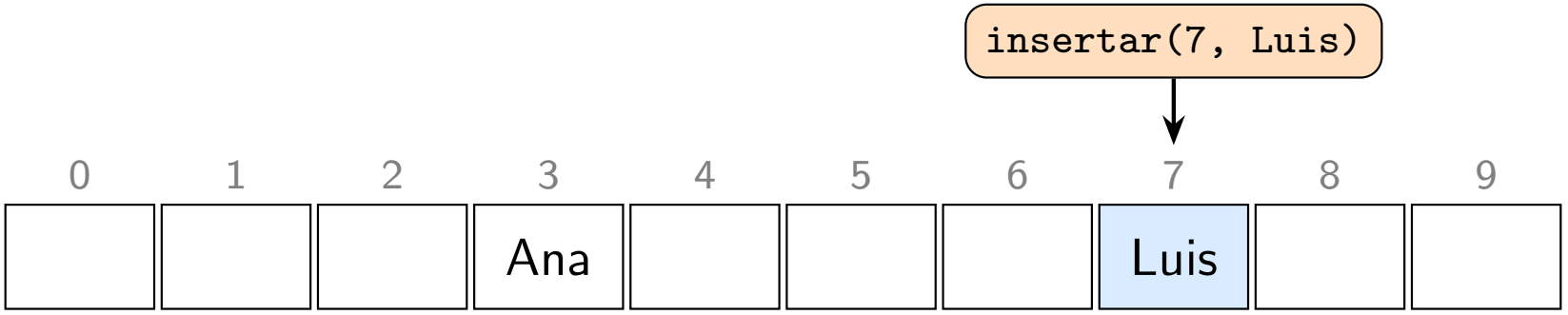
# La clave dice la casilla I

Un arreglo con una casilla por cada código posible: el registro del código $k$ va en la casilla $k$. Con códigos de 0 a 9, diez casillas, todas vacías al comenzar.

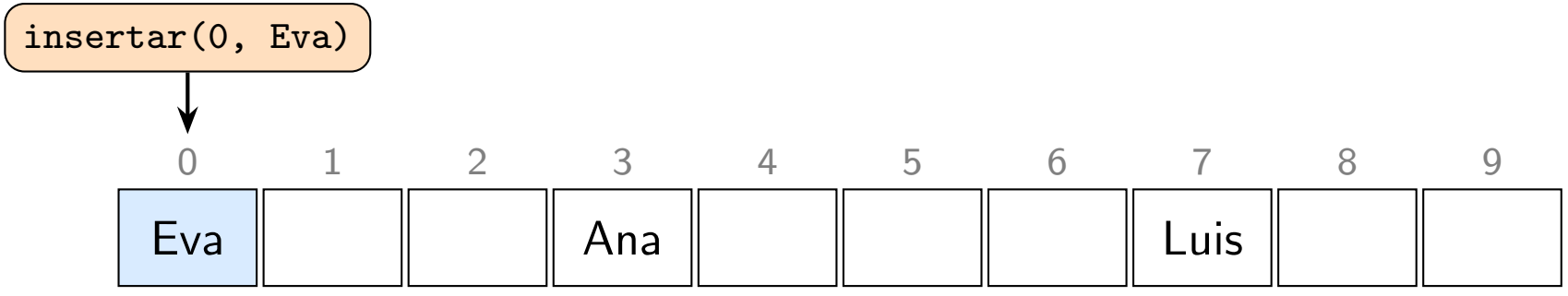



`insertar(3, Ana)`: la clave es 3, así que se escribe en la casilla 3. No se mira ninguna otra.

# La clave dice la casilla II




`insertar(7, Luis)`: casilla 7.



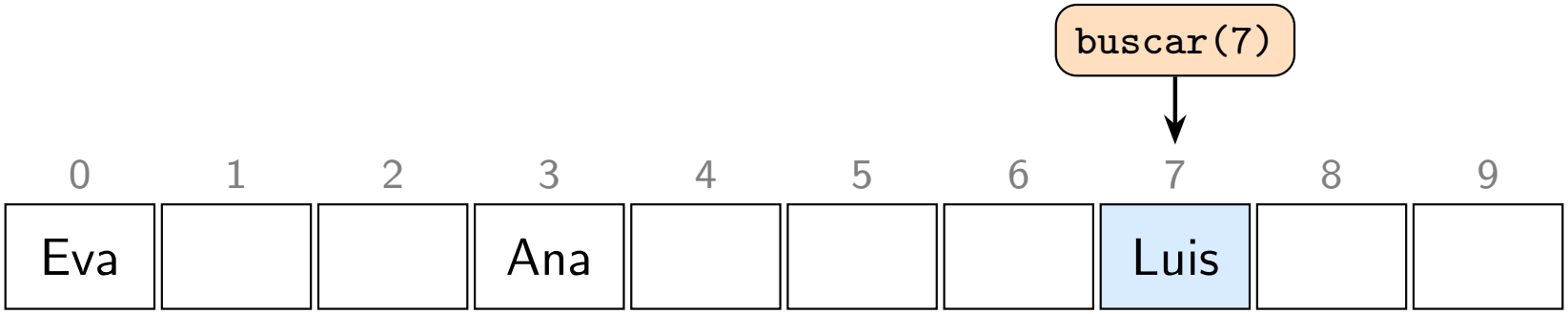

`insertar(0, Eva)`: casilla 0. El orden de llegada no importa; cada clave tiene su sitio.

# La clave dice la casilla III

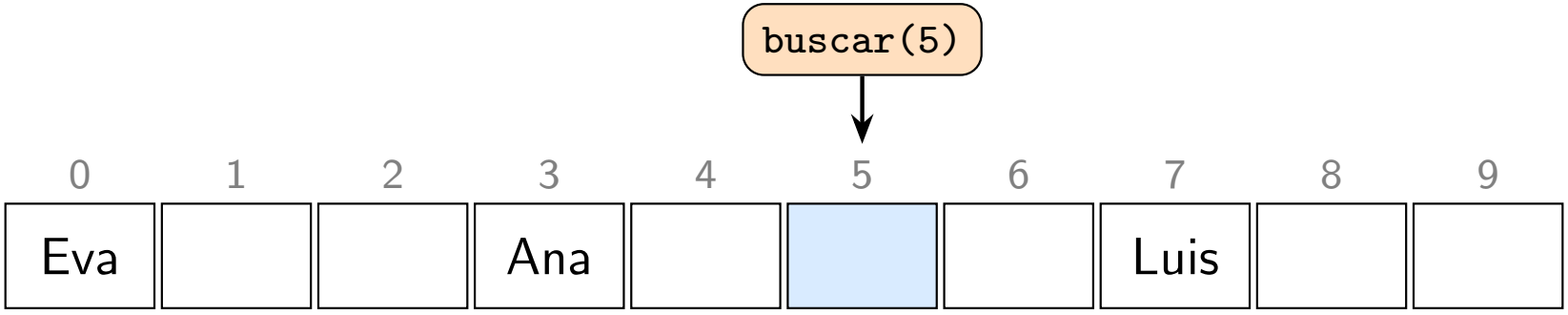
**Pregunta**

¿Cuántas casillas se leen para responder `buscar(7)`? ¿Y `buscar(5)`?

# La clave dice la casilla IV

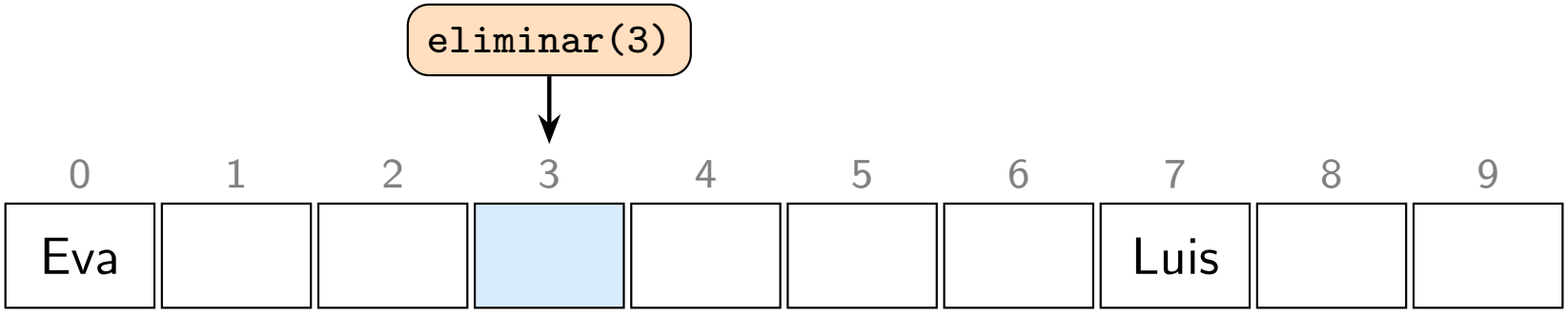



`buscar(7)`: una casilla, la 7, y ahí está Luis.




`buscar(5)`: también una sola. La casilla 5 está vacía, así que el 5 no está registrado, y no hace falta mirar las demás.

# La clave dice la casilla V




`eliminar(3)`: se vacía la casilla 3. Después, `buscar(3)` lee esa casilla y dice que no está.

Ninguna operación miró más de una casilla, y ninguna dependió de cuántos registros había.

# El TAD Diccionario I

## Diccionario (CLRS, introducción del capítulo 11)

Un conjunto dinámico de elementos, cada uno con una *clave* y datos que la acompañan, con tres operaciones:

- `insertar(x)`: agrega el elemento `x`.
- `buscar(k)`: devuelve el elemento de clave `k`, o `NULL` si no hay.
- `eliminar(k)`: quita el elemento de clave `k`.

## Tabla de direccionamiento directo (CLRS, §11.1)

Las claves salen de un universo $U = \{0, 1, \ldots, m-1\}$, con $m$ no muy grande, y dos elementos no tienen la misma clave. La tabla es un arreglo $T[0..m-1]$: la casilla $k$ apunta al elemento de clave $k$, o vale `NULL` si no hay ninguno.

En el laboratorio, $U$ son los códigos de 0 a 999 y $m = 1000$; el elemento es un registro con código y nombre.

# El diseño en pseudocódigo I

## Las tres operaciones (CLRS, §11.1)

```
buscar(T, k)
    devolver T[k]

insertar(T, x)
    T[x.clave] ← x

eliminar(T, k)
    T[k] ← NULL
```

O(1) O(1) O(1)

Cada una calcula una dirección y lee o escribe una casilla. No hay ciclo ni comparación de claves.

## Lo que la tabla supone

- Toda clave está entre 0 y $m - 1$. Una clave fuera de ese rango escribe fuera del arreglo.
- Las claves no se repiten: con dos registros de clave 3, el segundo ocupa la casilla del primero.

# TablaDirecta en C++ I

Un arreglo de punteros a `Registro` pedido con `new`, todos en `NULL` al comenzar.

```
struct Registro {
  int clave;
  char nombre[20];
};

class TablaDirecta {
private:
  Registro **T;       // T[k] apunta al registro de clave k, o vale
  ↪  NULL
  int m;              // tamano del universo de claves

public:
```

# TablaDirecta en C++ II

```
// Theta(m): todas las casillas empiezan vacias
TablaDirecta(int tam) {
  m = tam;
  T = new Registro*[m];
  int k = 0;
  while (k < m) {
    T[k] = NULL;
    k = k + 1;
  }
}
```

Arreglo de punteros tipo Registro

← nulo

# TablaDirecta en C++ III

```
// Theta(m): se revisan todas las casillas
~TablaDirecta() {
  int k = 0;
  while (k < m) {
    delete T[k];
    k = k + 1;
  }
  delete[] T;
}

// exige 0 <= clave < m; si la clave ya estaba, cambia el nombre
void insertar(int clave, const char *nombre) {
  assert(clave >= 0 && clave < m);
  if (T[clave] == NULL) {
    T[clave] = new Registro;
    T[clave]->clave = clave;
  }
  strcpy(T[clave]->nombre, nombre);
}
```

# TablaDirecta en C++ IV

```
// exige 0 <= clave < m; devuelve NULL si no hay registro
Registro *buscar(int clave) {
  assert(clave >= 0 && clave < m);
  return T[clave];
}

// exige 0 <= clave < m
void eliminar(int clave) {
  assert(clave >= 0 && clave < m);
  delete T[clave];
  T[clave] = NULL;
}
```

- La tabla es dueña de los registros: `insertar` los pide con `new` y `eliminar` los libera con `delete`. `delete` sobre `NULL` no hace nada, así que eliminar un código que no está es válido.
- `buscar` devuelve el puntero de la casilla: o el registro, o `NULL`.

# El laboratorio con la tabla I

```
int main() {
  TablaDirecta t(10);
  t.insertar(3, "Ana");
  t.insertar(7, "Luis");
  t.insertar(0, "Eva");
  t.imprimir();
  consultar(t, 7);
  consultar(t, 5);
  t.eliminar(3);
  consultar(t, 3);
  t.imprimir();
  return 0;
}
```

Los dos `imprimir()` escriben `0:Eva 1:- 2:- 3:Ana 4:- 5:- 6:- 7:Luis 8:- 9:-` y `0:Eva 1:- 2:- 3:- 4:- 5:- 6:- 7:Luis 8:- 9:-`.

# El laboratorio con la tabla II

| Operación | Casilla | Leídas | Salida |
|---|---|---|---|
| `insertar(3, Ana)` | 3 | 1 | |
| `insertar(7, Luis)` | 7 | 1 | |
| `insertar(0, Eva)` | 0 | 1 | |
| `imprimir()` | todas | 10 | la tabla, con Ana en la 3 |
| `consultar(7)` | 7 | 1 | `buscar(7): Luis` |
| `consultar(5)` | 5 | 1 | `buscar(5): no esta` |
| `eliminar(3)` | 3 | 1 | |
| `consultar(3)` | 3 | 1 | `buscar(3): no esta` |
| `imprimir()` | todas | 10 | la tabla, con la 3 vacía |

La salida es la misma que la de la lista. Lo que cambia es cuántas casillas se miran: una por operación, salvo `imprimir`, que recorre las $m$.

# Lo que cobra la tabla I

| Operación | Tiempo | Por qué |
|---|---|---|
| crear la tabla | $\Theta(m)$ | cada casilla empieza en `NULL` |
| `insertar(k, nombre)` | $\Theta(1)$ | la clave es la dirección: una casilla y, si está vacía, un `new` |
| `buscar(k)` | $\Theta(1)$ | se lee $T[k]$ |
| `eliminar(k)` | $\Theta(1)$ | un `delete` y $T[k] \leftarrow$ `NULL` |
| `imprimir`, destruir | $\Theta(m)$ | recorren todas las casillas, llenas o vacías |

Ptr 8 bytes = 64 bits  char 1 byte  clave 4 bytes  $m \times 8 \times (10 + y)$

## Espacio

$\Theta(m)$ punteros, uno por clave *posible*, más $\Theta(n)$ registros, uno por clave guardada. Como $n \leq m$, el total es $\Theta(m) = \Theta(|U|)$: lo fija el universo, no lo que se guarda. Con $m = 1000$ y punteros de 8 bytes, la tabla vacía ya ocupa 8 000 bytes.

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# El límite: un universo grande I

**Problema**

La puerta del laboratorio registra ahora por número de documento, de hasta 10 dígitos. En un día entran 40 personas.

**Pregunta**

Con la tabla directa, ¿cuánto ocupa la tabla?

# El límite: un universo grande II

**Con punteros de 8 bytes**

Una tabla de $m = 1000$ casillas ocupa 8 000 bytes. Con documentos de 10 dígitos, $m = 10^{10}$ y la tabla ocupa $8 \cdot 10^{10}$ bytes.

- Ochenta mil millones de bytes, 80 GB, para guardar 40 registros: se usarían 40 casillas de $10^{10}$.
- Crear la tabla cuesta $\Theta(m)$: poner $10^{10}$ casillas en `NULL` antes de la primera pregunta.

# ¿Entendimos el problema? I

**Entradas**

Los $n$ documentos registrados y una lista de consultas.

**Salidas**

`si` o `no` por cada consulta.

Con tres documentos registrados, 8920104417, 8920104452 y 8920103990:

| Consulta | Respuesta |
|---|---|
| 8920104452 | `si` |
| 8920104400 | `no` |
| 8920103990 | `si` |

**Pregunta**

La lista ocupa poco y la tabla directa busca rápido. ¿Alguna de las dos sirve aquí?

# ¿Entendimos el problema? II

| | Buscar | Espacio | Sirve cuando |
|---|---|---|---|
| Lista enlazada | $\Theta(n)$ | $\Theta(n)$ | $n$ es pequeño |
| Tabla directa | $\Theta(1)$ | $\Theta(\|U\|)$, con $\|U\| = 10^{10}$ | $\|U\|$ es pequeño |
| Lo que se quiere | cerca de $\Theta(1)$ | cerca de $\Theta(n)$ | $n$ y $\|U\|$ son grandes |

- Con la lista, $q$ consultas por documentos que no están recorren $q \cdot n$ nodos.
- Con la tabla directa, casi todas las casillas quedan vacías.

**Pregunta**

¿Se puede tener una tabla de pocas casillas para un universo enorme?

# Una tabla de 100 casillas I

## Tanteo

Se toma una tabla de 100 casillas y cada documento va a la casilla que dicen sus dos últimos dígitos.

| Documento | Casilla |
|---|---|
| 8920104417 | 17 |
| 8920104452 | 52 |
| 8920103990 | 90 |

Buscar 8920104452 es mirar la casilla 52: un acceso, como en la tabla directa, con 100 casillas en lugar de $10^{10}$.

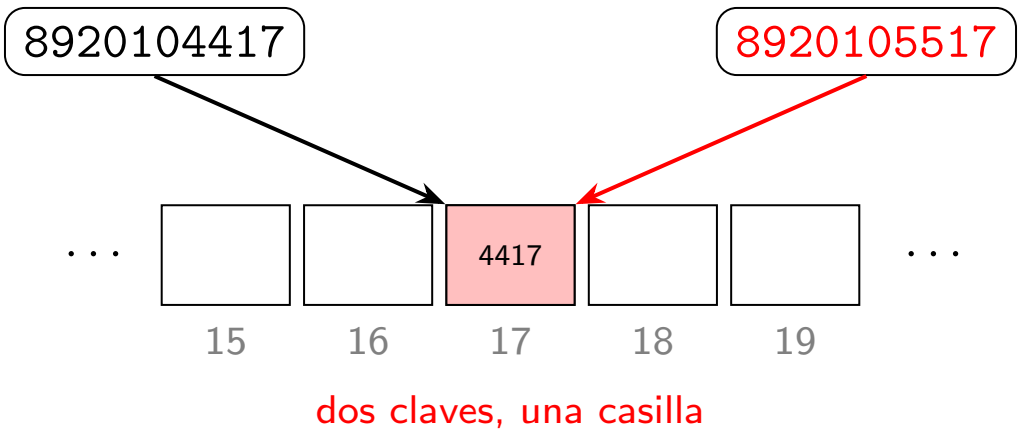
## Pregunta

Entra 8920105517. ¿A qué casilla va?

# Una tabla de 100 casillas II

**Choque**

A la 17, que ya tiene a 8920104417. Dos documentos distintos piden la misma casilla, y la casilla solo guarda uno.




- Hay $10^{10}$ documentos posibles y 100 casillas: a cada casilla le corresponden $10^8$ documentos. Ninguna regla para escoger la casilla evita los choques; lo que se puede hacer es repartirlos bien y saber qué hacer cuando ocurren.

# Función hash y colisión I

**Definición (CLRS, §11.2)**

Una *función hash* $h : U \to \{0, 1, \ldots, m-1\}$ asigna a cada clave del universo $U$ una casilla de una tabla $T[0..m-1]$. La clave $k$ se guarda en $T[h(k)]$, y $h(k)$ es el *valor hash* de $k$. Dos claves distintas con el mismo valor hash forman una *colisión*.

**En el tanteo**

$U$ son los documentos de 10 dígitos, $m = 100$ y $h(k)$ son los dos últimos dígitos de $k$. 8920104417 y 8920105517 colisionan en la casilla 17.

- Si $|U| > m$, dos claves del universo comparten casilla: las colisiones no se pueden evitar del todo.

# El método de la división I

**Definición (CLRS, §11.3)**

$h(k) = k$ mód $m$: la casilla es el residuo de dividir la clave entre el número de casillas. Los dos últimos dígitos son $k$ mód 100.

**Pregunta**

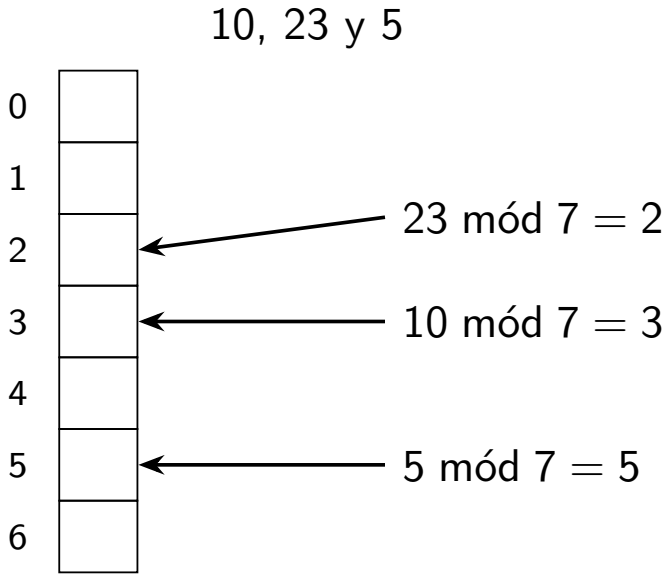
Con $m = 7$, ¿a qué casilla va cada clave?

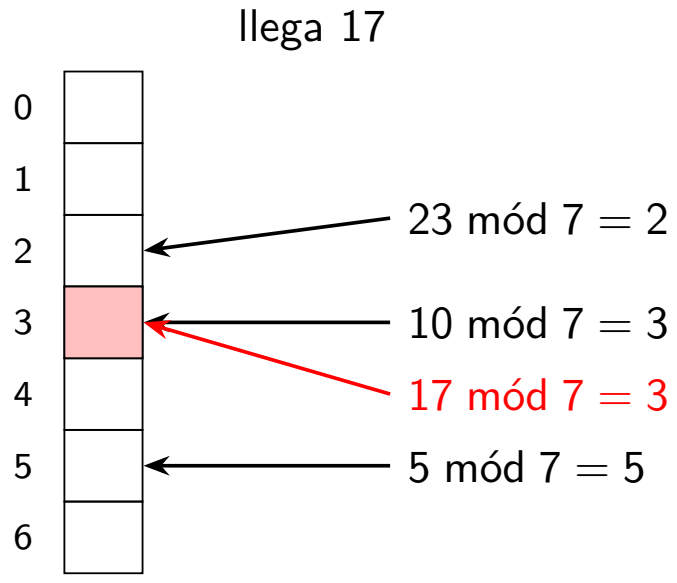
| $k$ | $k$ mód 7 | $h(k)$ |
|---|---|---|
| 10 | | |
| 23 | | |
| 5 | | |
| 17 | | |

# El método de la división II

| $k$ | $k$ mód 7 | $h(k)$ |
|---|---|---|
| 10 | $10 = 1 \cdot 7 + 3$ | 3 |
| 23 | $23 = 3 \cdot 7 + 2$ | 2 |
| 5 | $5 = 0 \cdot 7 + 5$ | 5 |
| 17 | $17 = 2 \cdot 7 + 3$ | 3 |

# El método de la división III




- 10 y 17 colisionan en la casilla 3.
- Calcular $h(k)$ es una división: $\Theta(1)$.
- En C++, `k % m` da el residuo solo si $k \geq 0$. Con claves negativas hay que tener cuidado, y aparece en el problema del juez.

# Claves que son palabras I

**Problema**

Para contar palabras de un texto, la clave es una cadena. Hay que convertirla en un número antes de tomar el residuo.

**Primer intento: sumar los códigos de los caracteres**

En C, cada `char` es un número: `'s'` es 115, `'o'` es 111, `'l'` es 108. Con $m = 13$: $115 + 111 + 108 = 334$ y 334 mód 13 $= 9$.

**Pregunta**

¿A qué casilla va `los`?

s o l
115 + 111 + 108 = 334

los

# Claves que son palabras II

## También a la 9

`los` tiene los mismos caracteres en otro orden y la suma no ve el orden. Toda palabra choca con sus anagramas: `gato` y `toga`, `amor` y `roma`.

$417 = 4 \times 10^2 + 1 \times 10^1 + 7 \times 10^0$

## Leer la cadena como un número en base $b$

Así como $417 = (4 \cdot 10 + 1) \cdot 10 + 7$, la cadena $c_0 c_1 \cdots c_{L-1}$ se lee como $(\cdots((c_0 \cdot b + c_1) \cdot b + c_2) \cdots) \cdot b + c_{L-1}$, y la casilla es ese número módulo $m$ (CLRS, §11.3). Cada carácter pesa distinto según su posición. Aquí $b = 31$.

# El polinomio, paso a paso I

$(31 \times 115 + 111) \times 31 + 108$

$31^2 \times 115 + 111 \times 31 + 108$

Con $b = 31$ y $m = 13$, se reduce módulo 13 en cada paso:

| Paso | Carácter | Código | $h \leftarrow (h \cdot 31 + c)$ mód 13 |
|---|---|---|---|
| 0 | | | $h = 0$ |
| 1 | `s` | 115 | $(0 \cdot 31 + 115)$ mód $13 = 11$ |
| 2 | `o` | 111 | $(11 \cdot 31 + 111)$ mód $13 = 452$ mód $13 = 10$ |
| 3 | `l` | 108 | $(10 \cdot 31 + 108)$ mód $13 = 418$ mód $13 = 2$ |

**Pregunta**

¿Por qué reducir en cada paso y no al final?

# El polinomio, paso a paso II

solecto $115 \times 31^9 + 67 \times 31^6 + 111 \times 31^5 + \ldots + 0 \times 31^0$

- Sin reducir, el número crece como $31^L$. Con 'a' $= 97$, una palabra de seis letras ya pasa de $97 \cdot 31^5 \approx 2{,}8 \cdot 10^9$, y un `int` llega hasta $2^{31} - 1 \approx 2{,}1 \cdot 10^9$. Reduciendo, $h < m$ siempre, y el residuo da el mismo resultado al final.

```c
// Lee la cadena como un numero en base 31 (Horner), reduciendo en
↪  cada paso.
int hashPolinomio(const char *s, int m) {
  int h = 0;
  int i = 0;
  while (s[i] != '\0') {
    h = (h * 31 + s[i]) % m;
    i = i + 1;
  }
  return h;
}
```

# El polinomio, paso a paso III

El programa calcula las dos funciones con $m = 13$:

| Palabra | Suma | Polinomio |
|---|---|---|
| `sol` | 9 | 2 |
| `los` | 9 | 3 |
| `gato` | 11 | 1 |
| `toga` | 11 | 12 |

- Con el polinomio los anagramas se separan. Otras palabras pueden chocar: con 13 casillas, es inevitable.
- Calcular el valor hash de una cadena de $L$ caracteres cuesta $\Theta(L)$: una vuelta por carácter.

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

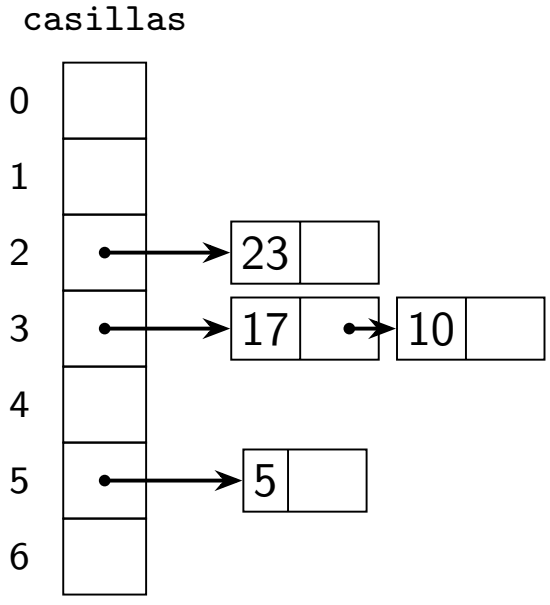
5 Para el juez

6 Ejercicios

# Una lista enlazada en cada casilla I

## Encadenamiento (CLRS, §11.2)

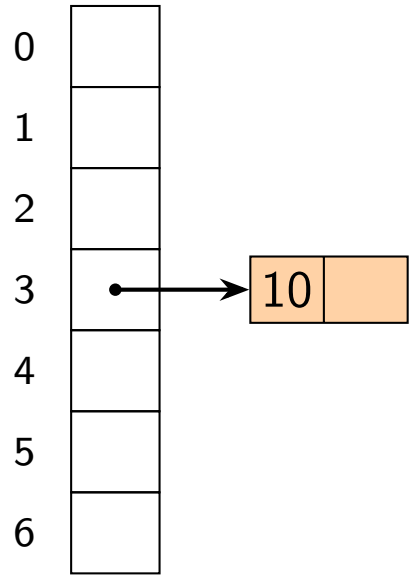
Cada casilla $T[i]$ guarda la cabeza de una lista enlazada con todas las claves $k$ tales que $h(k) = i$. Una casilla sin claves guarda `NULL`. Las colisiones se resuelven en la lista.



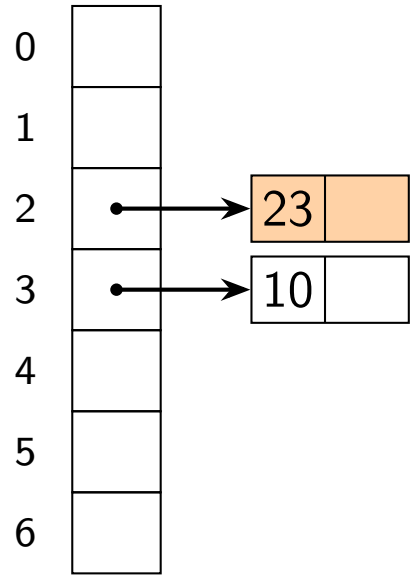

Las claves 10, 23, 5 y 17 con $m = 7$: 10 y 17 comparten la lista de la casilla 3. Cada nodo es el de la lista enlazada simple: la clave y el puntero al siguiente.

# Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ I

insertar 10: $h(10) = 3$




insertar 23: $h(23) = 2$

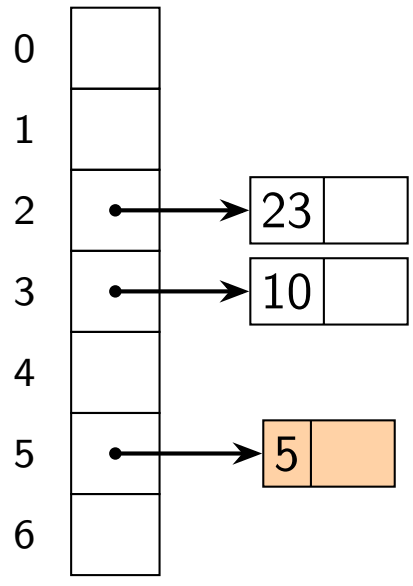



**Pregunta**

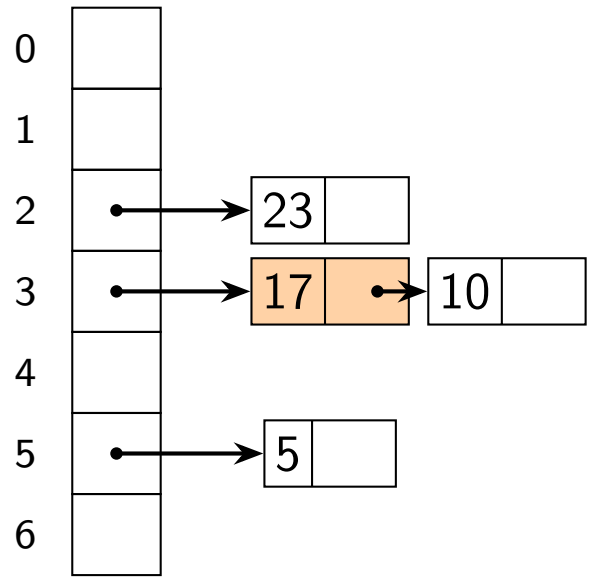
Después de insertar 5, llega 17 y $h(17) = 3$. ¿Va al frente de la lista de la casilla 3 o al final?

# Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ II

insertar 5: $h(5) = 5$




insertar 17: $h(17) = 3$

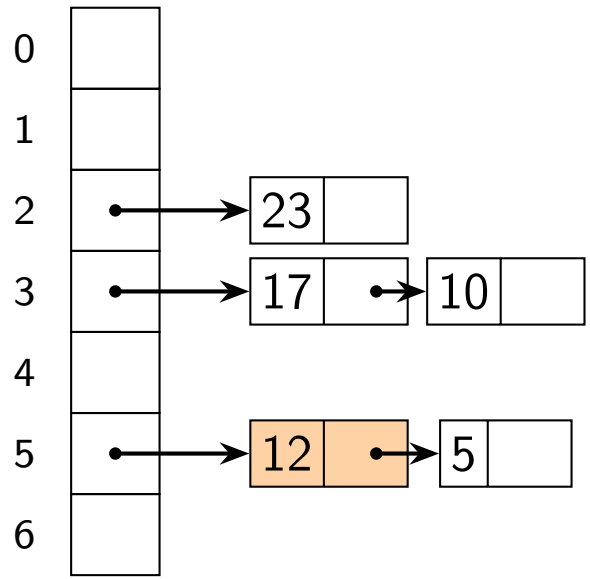



**Al frente**

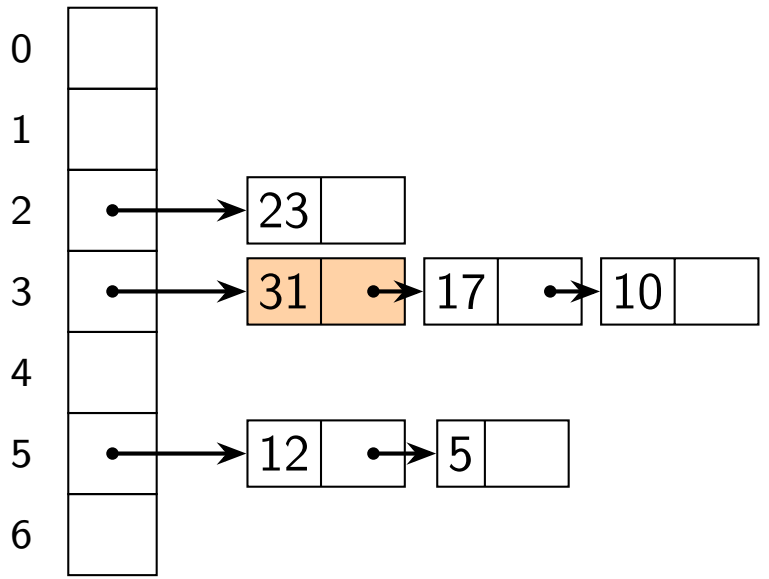
El 17 se cuelga antes del 10: el nuevo nodo apunta a la cabeza vieja y la casilla apunta al nuevo nodo. Son dos punteros, sin importar el largo de la lista. Ir al final pediría recorrerla.

# Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ III

insertar 12: $h(12) = 5$




insertar 31: $h(31) = 3$



# Paso a paso: 10, 23, 5, 17, 12, 31 con $m = 7$ IV

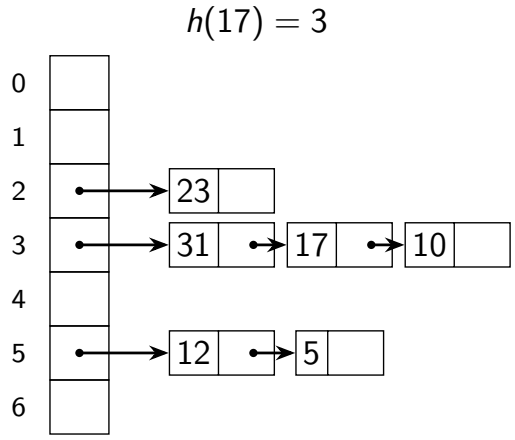
Lo que pasó en cada inserción:

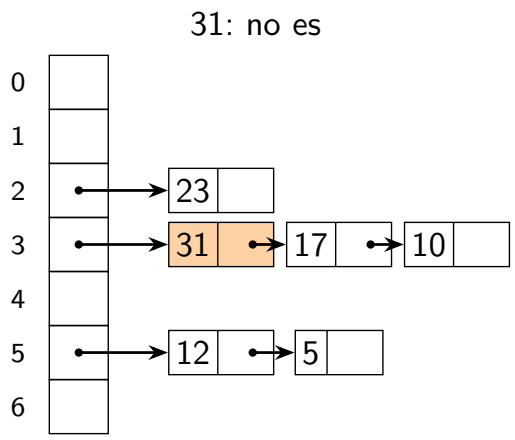
| Clave | $h(k)$ | Largo de su lista | Casillas vacías |
|---|---|---|---|
| 10 | 3 | 1 | 6 |
| 23 | 2 | 1 | 5 |
| 5 | 5 | 1 | 4 |
| 17 | 3 | 2 | 4 |
| 12 | 5 | 2 | 4 |
| 31 | 3 | 3 | 4 |

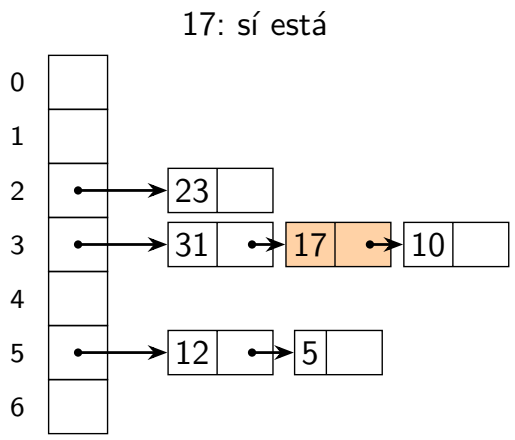
# Buscar I

## Pregunta

Buscar 17. ¿Qué lista se recorre y con cuántas claves se compara?




## Buscar 17

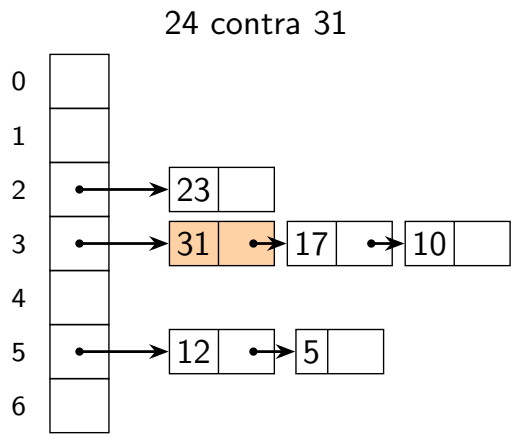
$h(17) = 3$. Se recorre la lista de la casilla 3: 31 no es, 17 sí. Dos comparaciones. Las listas de las casillas 2 y 5 no se miran.

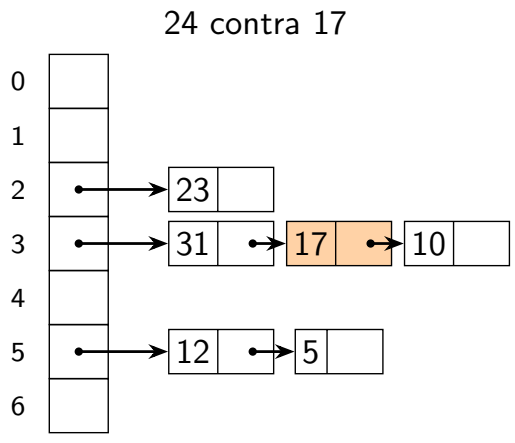
# Buscar II

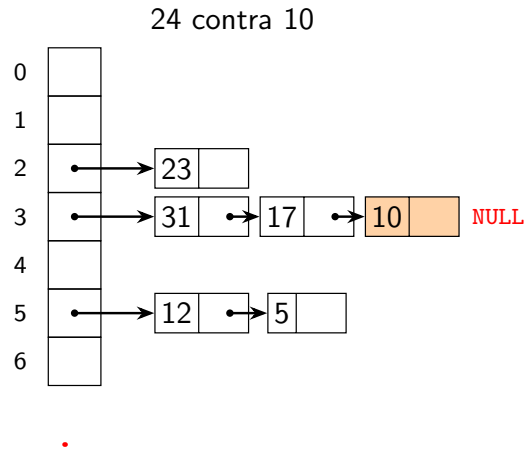
**Pregunta**

Buscar 24. ¿En qué casilla se busca y cuándo se sabe que no está?

# Buscar III



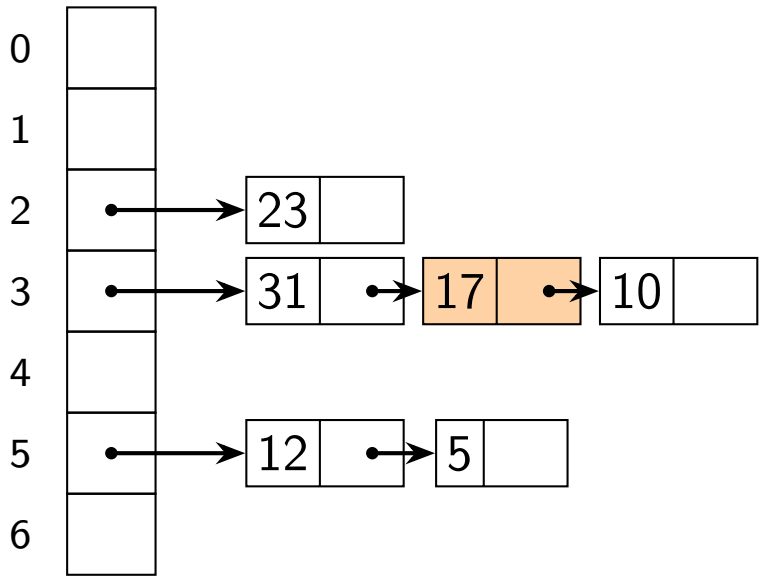

## Buscar 24

$h(24) = 24$ mód $7 = 3$. Se compara con 31, 17 y 10, y se llega a `NULL`: 24 no está. Tres comparaciones, el largo entero de la lista.

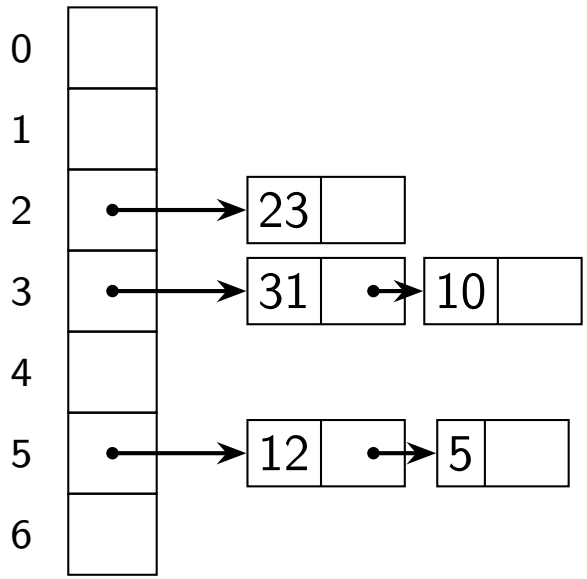
- Una búsqueda que falla recorre la lista completa de su casilla; una que acierta se detiene en la clave. El costo depende del largo de una lista, no del total de claves de la tabla.

# Eliminar I

antes: `anterior` en 31




después de eliminar 17



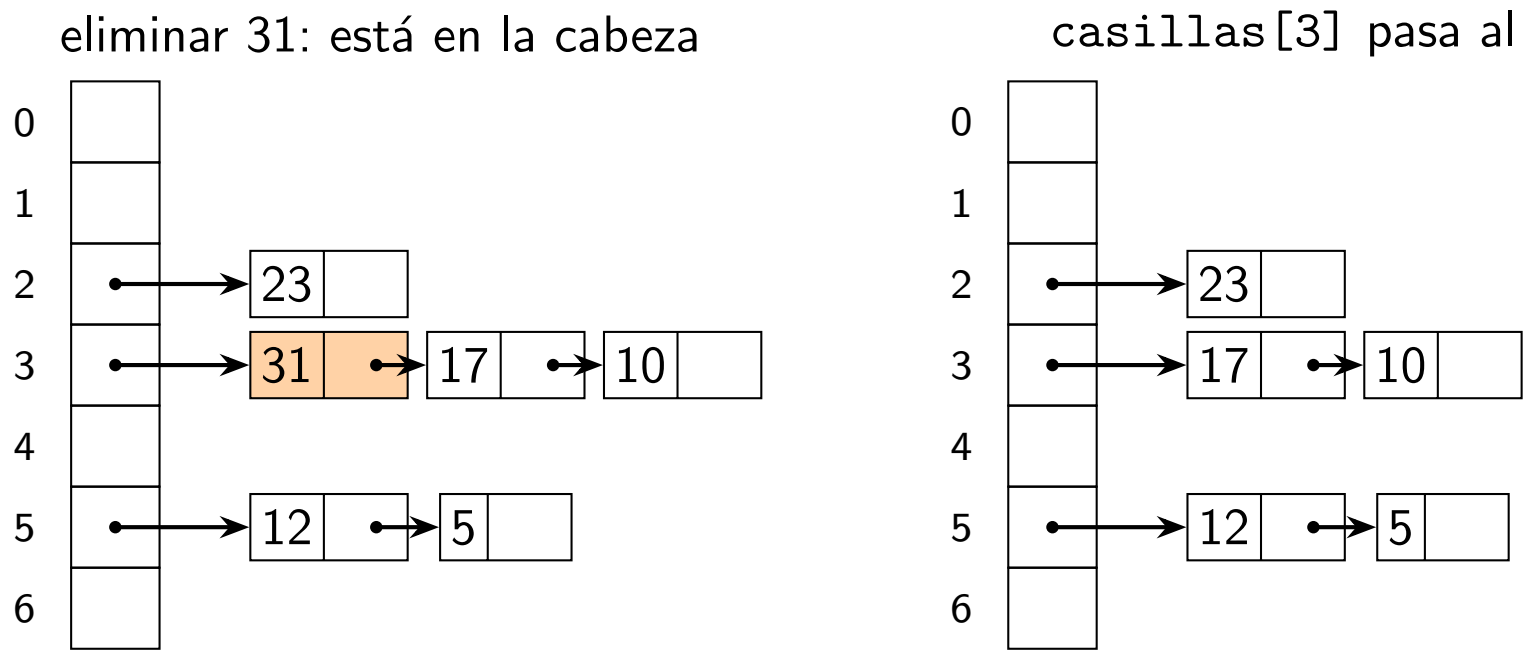
# Eliminar II

Eliminar 17 es eliminar en la lista enlazada: se recorre la lista de la casilla 3 con dos punteros, `anterior` y `actual`. Cuando `actual` llega al 17, el `siguiente` de 31 pasa a ser el 10 y el nodo del 17 se libera con `delete`.

**Caso límite**

Si la clave está en la cabeza, no hay `anterior`: la casilla misma pasa a apuntar al segundo nodo. Eliminar 31 deja `casillas[3]` en el 17.

# Eliminar III




Eliminar recorre la lista de su casilla hasta la clave: $\Theta(1+\alpha)$ en promedio. Desenlazar, en la cabeza o en medio, son dos punteros.

El alpha es el factor de carga n/m bajo la suposición de hashing uniforme

# La tabla en C++ I

```
struct Nodo {
  int clave;
  Nodo *siguiente;
};

class TablaHash {
private:
  Nodo **casillas;  // cabeza de la lista de cada casilla
  int m;            // numero de casillas
  int n;            // numero de claves guardadas

  // Metodo de la division. Exige k >= 0.
  int h(int k) {
    return k % m;
  }
```

# La tabla en C++ II

```
public:
  TablaHash(int tam) {
    m = tam;
    n = 0;
    casillas = new Nodo *[m];
    int i = 0;
    while (i < m) {
      casillas[i] = NULL;
      i = i + 1;
    }
  }
```

El constructor deja las $m$ cabezas en `NULL`: $\Theta(m)$.

# La tabla en C++ III

```
// Recorre la lista de h(k) hasta hallar k o llegar a NULL.
bool buscar(int k) {
  Nodo *actual = casillas[h(k)];
  while (actual != NULL && actual->clave != k) {
    actual = actual->siguiente;
  }
  return actual != NULL;
}
```

```
// Cuelga k al frente de su lista. Exige que k no este.
void insertar(int k) {
  int i = h(k);
  Nodo *nuevo = new Nodo;
  nuevo->clave = k;
  nuevo->siguiente = casillas[i];
  casillas[i] = nuevo;
  n = n + 1;
}
```

# La tabla en C++ IV

```
// Desenlaza el nodo de k. Devuelve si lo encontro.
bool eliminar(int k) {
  int i = h(k);
  Nodo *anterior = NULL;
  Nodo *actual = casillas[i];
  while (actual != NULL && actual->clave != k) {
    anterior = actual;
    actual = actual->siguiente;
  }
  bool estaba = actual != NULL;
  if (estaba) {
    if (anterior == NULL) {
      casillas[i] = actual->siguiente;
    } else {
      anterior->siguiente = actual->siguiente;
    }
    delete actual;
    n = n - 1;
  }
  return estaba;
}
```

# La tabla en C++ V

```
~TablaHash() {
  int i = 0;
  while (i < m) {
    while (casillas[i] != NULL) {
      Nodo *muerto = casillas[i];
      casillas[i] = casillas[i]->siguiente;
      delete muerto;
    }
    i = i + 1;
  }
  delete[] casillas;
}
```

- El destructor libera nodo por nodo cada lista y después el arreglo de cabezas: cada `new` tiene su `delete` y el `new[]` su `delete[]`.
- `insertar` no revisa si la clave ya está. Si las claves pueden repetirse, se llama a `buscar` antes.

# El ejemplo, corriendo I

Un `main` crea `TablaHash t(7)`, inserta 10, 23, 5, 17, 12 y 31 imprimiendo $h(k)$, imprime la tabla, busca 17 y 24, elimina 17 y pide el largo de la casilla 3.

**Salida**

`h(10) = 3`, `h(23) = 2`, `h(5) = 5`, `h(17) = 3`, `h(12) = 5`, `h(31) = 3`, y la tabla:

```
0:  1:  2: 23  3: 31 17 10  4:  5: 12 5  6:
```

Después, `buscar 17: 1`, `buscar 24: 0`, `eliminar 17: 1`, `largo de la casilla 3: 2` y `n = 5, m = 7`.

Las listas salen como en los dibujos: la última clave insertada en cada casilla queda de primera.

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# El factor de carga I

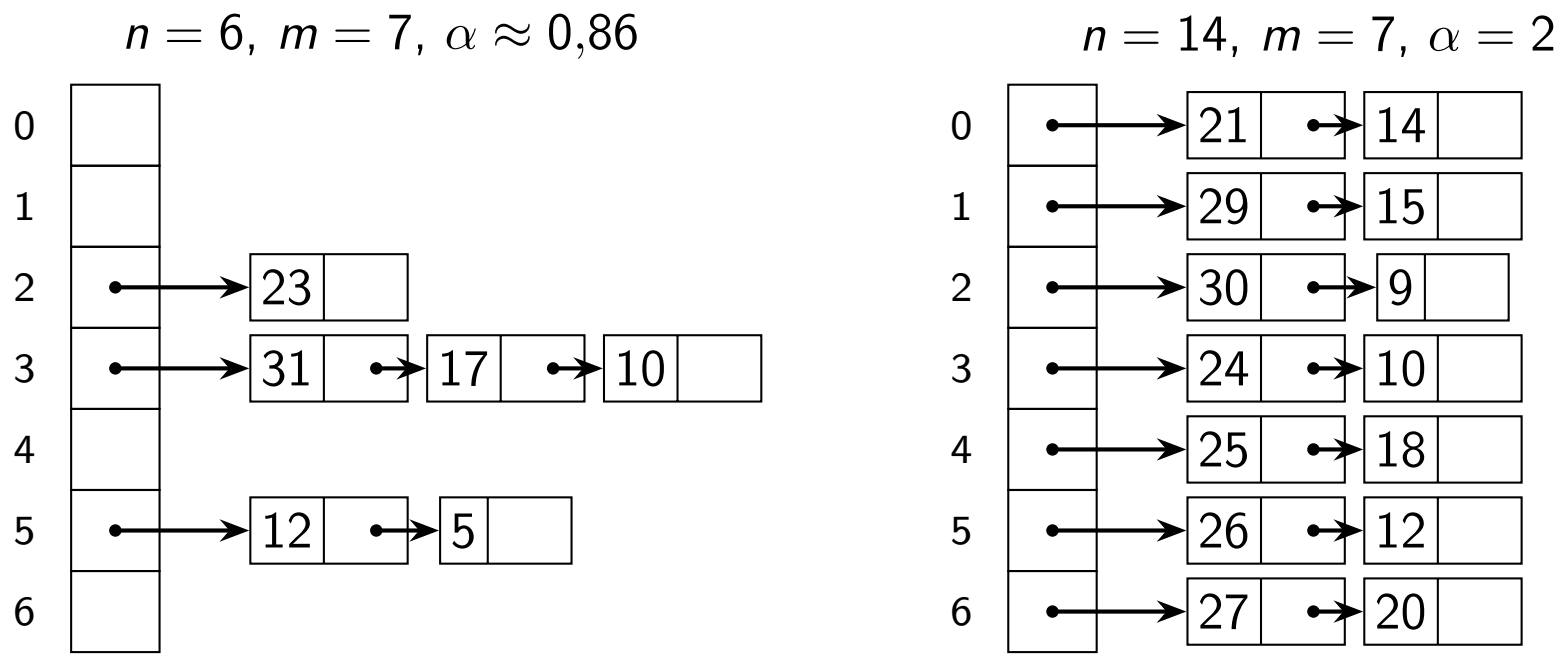
**Definición (CLRS, §11.2)**

Con $n$ claves en una tabla de $m$ casillas, el *factor de carga* es $\alpha = n/m$: el largo promedio de las listas.

**En el ejemplo**

Con las seis claves, $\alpha = 6/7 \approx 0{,}86$. Las listas miden 0, 0, 1, 3, 0, 2 y 0: suman 6, y su promedio es $6/7$. Tras eliminar 17, $\alpha = 5/7 \approx 0{,}71$.

# El factor de carga II




A la izquierda, listas de 0, 0, 1, 3, 0, 2 y 0: promedio 6/7. A la derecha la tabla está llena y bien repartida: cada lista mide exactamente $\alpha = 2$, y $\alpha$ puede pasar de 1.

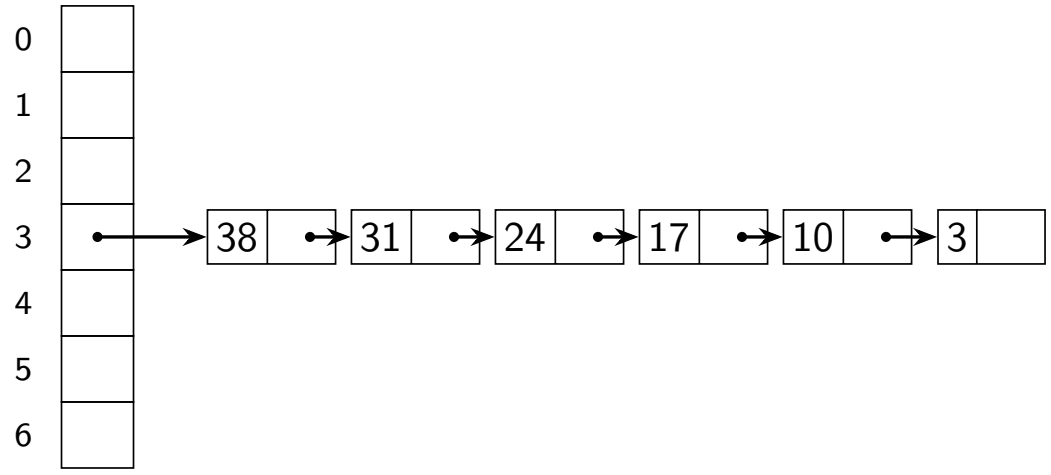
# El factor de carga III

**Pregunta**

$\alpha$ es el promedio. ¿Cuánto puede medir la lista más larga?

# El peor caso: todas a la misma casilla I

Las claves 3, 10, 17, 24, 31 y 38 con $m = 7$: todas dejan residuo 3.




- Seis casillas vacías y una lista con las $n = 6$ claves. $\alpha$ sigue siendo $6/7$.
- Buscar 45, con $h(45) = 3$, compara con las seis antes de llegar a `NULL`. La tabla se comporta como la lista enlazada del comienzo: buscar y eliminar cuestan $\Theta(n)$.
- Insertar al frente sigue costando $\Theta(1)$.

# El caso promedio I

**Hashing uniforme independiente (CLRS, §11.2)**

Se supone que cada clave cae en cualquiera de las $m$ casillas con la misma probabilidad, sin importar dónde cayeron las demás.

- Con ese supuesto, la lista de una casilla mide en promedio $\alpha$. Una búsqueda que falla recorre una lista entera; una que acierta, en promedio, cerca de la mitad de una lista más el nodo de la clave.
- CLRS (teoremas 11.1 y 11.2) concluye que las dos cuestan, en promedio, $\Theta(1+\alpha)$: el 1 es calcular $h(k)$ y llegar a la casilla, y $\alpha$ es el recorrido.

**Pregunta**

Si $n$ crece y $m$ se deja fijo, ¿qué pasa con $1+\alpha$? ¿Y si $m$ crece al mismo ritmo que $n$?

# El caso promedio II

- Con $m$ fijo, $\alpha = n/m$ crece con $n$ y buscar vuelve a ser lineal, solo que $m$ veces más rápido que la lista.
- Con $m$ proporcional a $n$, por ejemplo $m \geq n/2$, se tiene $\alpha \leq 2$ y $1 + \alpha \leq 3$: buscar y eliminar cuestan $O(1)$ en promedio.

# Lo que cobra la tabla hash con encadenamiento I

| Operación | Promedio | Peor caso | Por qué |
|---|---|---|---|
| $h(k)$, clave entera | $\Theta(1)$ | $\Theta(1)$ | una división |
| $h(s)$, cadena de $L$ | $\Theta(L)$ | $\Theta(L)$ | una vuelta por carácter |
| `insertar(k)` | $\Theta(1)$ | $\Theta(1)$ | se cuelga al frente de su lista |
| `buscar(k)` | $\Theta(1+\alpha)$ | $\Theta(n)$ | se recorre una lista; en el peor caso, todas las claves están en ella |
| insertar sin repetir | $\Theta(1+\alpha)$ | $\Theta(n)$ | primero se busca |
| `eliminar(k)` | $\Theta(1+\alpha)$ | $\Theta(n)$ | se busca con `anterior`; desenlazar es $\Theta(1)$ |
| `largo(i)` | $\Theta(1+\alpha)$ | $\Theta(n)$ | se cuentan los nodos de una lista |
| `imprimir`, destructor | $\Theta(m+n)$ | $\Theta(m+n)$ | se visitan las $m$ casillas y los $n$ nodos |
| constructor | $\Theta(m)$ | $\Theta(m)$ | $m$ cabezas en `NULL` |

# Lo que cobra la tabla hash con encadenamiento II

## Espacio

$\Theta(m+n)$: el arreglo de $m$ cabezas y un nodo por clave, con la clave y un puntero. Ninguna operación pide memoria aparte que crezca con $n$.

| | Buscar, promedio | Buscar, peor caso | Espacio |
|---|---|---|---|
| Lista enlazada | $\Theta(n)$ | $\Theta(n)$ | $\Theta(n)$ |
| Tabla directa | $\Theta(1)$ | $\Theta(1)$ | $\Theta(\|U\|)$ |
| Tabla hash, $m$ proporcional a $n$ | $O(1)$ | $\Theta(n)$ | $\Theta(n)$ |

# Escoger $m$ I

**Pregunta**

Se insertan las 1000 claves $4, 8, 12, \ldots, 4000$ en una tabla de $m = 64$ casillas. $\alpha = 1000/64 \approx 15{,}6$. ¿Cuánto mide la lista más larga?

# Escoger $m$ II

Un programa inserta $k = paso \cdot i$ con $i = 1, \ldots, 1000$ y mide cada lista con `largo`:

| $m$ | paso | Casillas vacías | Lista más larga | $\alpha$ |
|---|---|---|---|---|
| 64 | 1 | 0 | 16 | 15,6 |
| 64 | 4 | 48 | 63 | 15,6 |
| 61 | 4 | 0 | 17 | 16,4 |
| 61 | 61 | 60 | 1000 | 16,4 |

# Escoger $m$ III

- $64 = 2^6$, y $k$ mód $2^6$ son los últimos 6 bits de $k$. Un múltiplo de 4 termina en 00: solo quedan 16 casillas posibles, las múltiplos de 4, y las otras 48 nunca se usan. Las listas miden cuatro veces $\alpha$.
- Con el primo 61 los múltiplos de 4 recorren todos los residuos: ninguna casilla vacía y listas de 16 o 17.
- Ningún $m$ resiste un patrón hecho a su medida: con paso 61, las 1000 claves caen en la casilla 0 de la tabla de 61.

**Regla (CLRS, §11.3)**

Con el método de la división se escoge un primo que no esté cerca de una potencia de 2. Una potencia de 2 o de 10 usa solo los últimos bits o los últimos dígitos de la clave, como la tabla de 100 casillas con los documentos.

1. Si uso un valor multiplo de 2, los ultimo k bits van a determinar el residuo, es mas barato hacerlo en la CPU pero nos dan mas casillas repetidas

2. Con un numero primo, dado que no tiene patrones (divisores enteros distintos a 1 o si mismo) se van a distribuir los residuos

3. No importa la valor de m que escoja, van a existir situaciones donde van a caer en las mismas casillas

4. Un buen m se busca que sea un numero primo lejos de un valor de multiplo de 10 o de 2 para evitar dependa de los bits

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Para el juez: UVa 484, The Department of Redundancy Department I

**UVa 484, The Department of Redundancy Department**,
`https://onlinejudge.org/external/4/484.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=425`

## Qué pide

La entrada es una secuencia de enteros positivos, negativos o cero, tan larga como se quiera, hasta el fin del archivo. Por cada entero distinto se imprime una línea con el entero y cuántas veces apareció, en el orden en que apareció por primera vez.

## ¿Entendimos el problema?

Con 3 1 2 2 1 3 5 3 3 2 sale 3 4, 1 2, 2 3 y 5 1. El 3 va primero porque fue el primero en llegar, no porque sea el que más se repite.

# Para el juez: UVa 484, The Department of Redundancy Department II

## Cómo atacarlo con una tabla hash

Una tabla hash con encadenamiento cuyo nodo guarda la clave y un contador. Por cada número se busca en su lista: si está, se suma uno; si no, se cuelga un nodo con cuenta 1 y el número se anota en un arreglo de claves en orden de llegada. Al final se recorre ese arreglo y se busca la cuenta de cada una. Con $m = 100\,003$, primo, cada número cuesta $\Theta(1 + \alpha)$ en promedio.

## Las trampas

- Los negativos: en C++, `-7 % 5` da $-2$. Con `k % m` como casilla, un negativo escribe fuera del arreglo. La casilla es `((k % m) + m) % m`.
- No se sabe cuántos números vienen, así que no hay $n$ para dimensionar: $m$ se fija grande desde el comienzo.
- $-100\,003$, 0 y $100\,003$ caen en la misma casilla y son claves distintas: la lista compara la clave, no la casilla.

# Contenido I


1 TAD tabla de direccionamiento directo

2 Pocas claves, universo enorme: la función hash

3 Encadenamiento

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Ejercicio en parejas: a mano I

Inserten, en este orden, 22, 5, 16, 38, 27, 49 y 3 en una tabla con encadenamiento de $m = 11$ casillas, colgando al frente.

## Las preguntas

1. Dibujen la tabla. ¿Cuánto vale $\alpha$ y cuánto mide la lista más larga?
2. ¿Cuántas comparaciones hace buscar 60?
3. Repitan con $m = 10$. ¿Qué tabla queda mejor repartida, y por qué no es la del primo?

# Ejercicio en parejas: a mano II

## Lo que debe salir

1. Con $m = 11$: 22 va a la casilla 0, 3 a la 3, y 5, 16, 38, 27 y 49 a la 5, con la lista `49 27 38 16 5`. $\alpha = 7/11 \approx 0{,}64$ y la lista más larga mide 5.
2. 60 mód 11 = 5: cinco comparaciones y `NULL`. No está.
3. Con $m = 10$ cada clave va a su propia casilla: 2, 5, 6, 8, 7, 9 y 3. Las claves 5, 16, 27, 38 y 49 van de 11 en 11, un patrón hecho a la medida de $m = 11$. El primo protege contra los patrones comunes, no contra todos.

# Ejercicio en parejas: dos que suman $x$ I

Dado un arreglo de $n$ enteros no negativos y un número $x$, decir si hay dos casillas distintas cuyos valores suman $x$.

## Las preguntas

1. Con $a = \langle 8, 3, 11, 6, 2, 15 \rangle$, ¿qué responde con $x = 9$, $x = 21$ y $x = 12$?
2. ¿Cuánto cuesta probar todas las parejas?
3. Al llegar a $a[i]$, ¿qué número habría que haber visto antes? ¿Qué estructura responde eso rápido?

# Ejercicio en parejas: dos que suman $x$ II

## Lo que debe salir

1. $x = 9$: sí, $3 + 6$. $x = 21$: sí, $6 + 15$. $x = 12$: no; $6 + 6$ no cuenta, porque hay un solo 6. El programa imprime `1`, `1` y `0`.
2. Todas las parejas son $n(n-1)/2$: $\Theta(n^2)$.
3. $x - a[i]$. Se busca en la tabla de los ya vistos y después se guarda $a[i]$. Buscar antes de guardar es lo que impide usar el 6 dos veces. Son $n$ búsquedas e inserciones de $\Theta(1 + \alpha)$ en promedio: $\Theta(n)$ con $m$ proporcional a $n$, más $\Theta(n)$ de espacio aparte.

# Propuestos para la casa: tabla directa I

1. **Conjunto de banderas.** Implemente un conjunto de enteros de 0 a $m-1$ con un arreglo de `bool`: `agregar`, `quitar`, `pertenece`, `unir` e `intersecar`. Dé el costo de cada una.

2. **¿Hay repetidos?** Dado un arreglo de $n$ enteros, todos entre 0 y 999, diga si algún valor aparece dos veces. Escriba la versión que compara cada par y la que usa una tabla de 1000 casillas, y compare sus costos con $\langle 412, 87, 903, 15, 87, 230\rangle$ y con $\langle 5, 999, 0, 314\rangle$.

3. **Un universo que no empieza en 0.** Los códigos del laboratorio van ahora de 1000 a 1999. Escriba `buscar` e `insertar` con una tabla de 1000 casillas. Después, los códigos válidos son solo los pares entre 0 y 1998: ¿cuántas casillas hacen falta, y qué cuenta convierte un código en su casilla?

# Propuestos para la casa: tablas hash I

4. **Distintos.** Escriba una función que cuente cuántos valores distintos hay en un arreglo de $n$ enteros no negativos con la tabla hash. ¿Cuánto cuesta? ¿Y ordenando primero?

5. **Tabla de palabras.** Cambie la clave de `TablaHash` a una cadena de hasta 20 letras, con `hashPolinomio` como $h$. ¿Qué cambia en `buscar`?

6. **El peor caso a la medida.** Para $m = 13$, escriba ocho claves que caigan todas en la casilla 4, y cuente cuántas comparaciones hace buscar una clave que no está.

7. **Repartir documentos.** Genere 2000 documentos de 10 dígitos que comparten los seis primeros y mida, con `largo`, la lista más larga y las casillas vacías con $m = 100$, $m = 128$ y $m = 101$.

8. **Eliminar con la lista doble.** Si cada casilla fuera una lista doblemente enlazada y se tuviera el nodo en la mano, ¿cuánto costaría eliminar?

# Propuesto para el juez: UVa 442 I

**UVa 442, Matrix Chain Multiplication**,
`https://onlinejudge.org/external/4/442.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=383`

## Qué pide

Hasta 26 matrices, cada una con una letra, y expresiones con paréntesis completos, como `(A(BC))`. Multiplicar $p \times q$ por $q \times r$ cuesta $p \cdot q \cdot r$. Por cada expresión, el total o `error`.

## Cómo atacarlo

Las dimensiones van en una tabla directa de 26 casillas, la de la letra $c$ en `c - 'A'`: leerla cuesta $\Theta(1)$. La expresión se recorre con una pila: una letra apila su matriz y un `)` saca dos y apila el producto. $\Theta(L)$ por expresión.

## La trampa

El primero que sale de la pila es la matriz de la *derecha*.

# Pistas para los propuestos I

1. Las tres primeras leen o escriben una casilla, $\Theta(1)$; unir e intersecar recorren las $m$ casillas con `||` y `&&`, $\Theta(m)$.
2. La casilla $v$ guarda una bandera: si ya apareció el valor $v$. Comparar cada par es $\Theta(n^2)$ en el peor caso; la tabla cuesta $O(n+m)$ con $m = 1000$, y $\Theta(m)$ de espacio aparte.
3. Restar 1000 deja las claves entre 0 y 999. Con los pares, la cuenta que se busca también es una sola operación.
4. Se busca cada valor y, si no estaba, se inserta y se suma uno. $\Theta(n)$ en promedio con $m$ proporcional a $n$; ordenando, $\Theta(n \log n)$ y un recorrido que compara vecinos.
5. El nodo guarda un arreglo de `char`; la comparación `actual->clave != k` pasa a ser `strcmp`, y cada comparación cuesta lo que la cadena.
6. Sirve cualquier clave $4 + 13j$. Buscar una que no está recorre las ocho.
7. Con 100 casillas solo cuentan los dos últimos dígitos, y con 128, los últimos 7 bits. Compare con el primo.
8. Con `anterior` en cada nodo no hay que buscar al de antes: desenlazar es $\Theta(1)$ (CLRS, §11.2).

# Resumen I

- El TAD Diccionario tiene `insertar`, `buscar` y `eliminar`. Con claves en $U = \{0, \ldots, m-1\}$, la tabla de direccionamiento directo las hace en $\Theta(1)$: la clave es la casilla. Ocupa $\Theta(|U|)$ aunque guarde pocos elementos.
- Una función hash lleva cada clave a una de $m$ casillas. Con el método de la división, $h(k) = k$ mód $m$; una cadena se lee como un número en base 31, reduciendo en cada paso.
- Las colisiones son inevitables cuando el universo es mayor que la tabla. Con encadenamiento, cada casilla es la cabeza de una lista enlazada: insertar al frente cuesta $\Theta(1)$, y buscar y eliminar recorren una sola lista.
- Con $\alpha = n/m$, buscar y eliminar cuestan $\Theta(1 + \alpha)$ en promedio y $\Theta(n)$ en el peor caso. El espacio es $\Theta(m + n)$.
- $m$ se escoge primo y lejos de una potencia de 2: con $m = 64$ y claves múltiplos de 4, 48 casillas quedaron vacías.

**Lo que sigue**

El contenedor `map` de la biblioteca, que guarda pares de clave y valor, y problemas que se resuelven con él.

# Referencias I




T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 11, tablas hash: introducción, operaciones de diccionario; §11.1 tablas de direccionamiento directo; §11.2 tablas hash, encadenamiento y su análisis; §11.3 funciones hash, método de la división.



R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 15, *Hashing and Collision*: §15.2 tablas hash; §15.4.1 método de la división; §15.5.2 resolución de colisiones por encadenamiento.



UVa Online Judge. Problema 484, *The Department of Redundancy Department*: `https://onlinejudge.org/external/4/484.pdf`.



UVa Online Judge. Problema 442, *Matrix Chain Multiplication*: `https://onlinejudge.org/external/4/442.pdf`.